

Overview of Database System

Database System = Databases + DBMS

- Model real-world enterprises/applications
- Tables/entities/relations (e.g., students, courses)
- Relationships
- A relational DB is a collection of tables

Database Management System (DBMS): software that manages databases

- Facilitate accesses to databases
- Accesses = queries and updates = reads and writes

Using files: • Application handles data accesses directly • Buffering • Writing data to disk • Optimizing performance • Protecting data from inconsistencies (concurrency issues) • Security and access control • Special application code for different queries, and HW/SW • Custom recovery code	Using DBMS: • Application calls DBMS for data access • DBMS handles issues related to data access • Data independence • Buffering, performance optimization, consistency, security... • Easier application development • Universal interface (e.g., SQL) • Same code for different platforms • No need to worry about recovery
---	---

Tables - relations

- A table consists of rows (aka records, tuples) and columns
- A row consists of fields, each column has a unique name
- A column includes all fields under a specific attribute
- Tables are instance of schemas

Schemas

- DBMS enforces schemas defined by database designer
- Schema is “metadata” that defines what a table looks like: column definition, constraints

Operations: Modifications - insert update delete

Operations: Search (Read) - SELECT * FROM table WHERE...

Data Independence

- Application (i.e., user of DBMS) views data under the relational model
- DBMS uses physical, in-memory or in-storage representations of data

Application = Application that uses a DBMS to access databases (typically SQL)

DBMS = Software system that handles access to databases (records, tables) in storage devices

OS = Tools for building a DBMS: file systems, synchronization primitives...

Hardware = CPU, memory, SSD, disks, network...

Application-DBMS Interface

Embedded - Application uses DBMS functionality via direct API function calls

Tiered client-server

- Application establishes a connection with the DBMS's client communications manager, via JDBC/ODBC/etc.
- Connect to some server(s) that connect to the database server

Embedded DBMS

- DBMS as a shared library, no server
- DBMS and application live in the same address space

Tiered Client-Server Architecture

- Application and DBMS reside in separate machines - multiple servers

Process Manager

- Queries (SQL command) arrive through application DBMS the connection/API calls
- DBMS must assign a thread of computation to handle the commands/transactions
- Admission control
 - The DBMS and server have a limited of resources
- Queries reaches query processor
 - Check authorization, generate a query plan
- Plan executor
 - series of general-purpose “operators” for any query
 - Call into the transactional storage manager

Transactional Storage Manager - storage engine

- Manage all data accesses and manipulations
- Key component for online transaction processing (OLTP)
- Guarantee data integrity

Hardware Fundamentals for Database System

Registers L1 Cache L2 Cache L3 Cache. Main memory - volatile DRAM

Flash/disk - non volatile

Hard disk - only single sector accesses are atomic. Caches: internal, invisible to software

NAND Flash memory - a type of non-volatile computer storage

- Basic building block: floating gate transistors (cells)
 - data stored in cells, number of bits determined by voltage levels
- Flash memory types
 - Single level cell - 1 bit per cell. Multi level cell - 2 bits per cell
- Organized in blocks and pages
 - Can read or write at page level, Modification will erase and rewrite block

Flash Memory based Solid State Drives (SSDs)

- Block interface (like disk) – sectors: - emulate base on SSD blocks and pages
- Flash Translation Layer FTL - mapping table, garbage collection, bad block management
- RAM - caching data, caching Ftl mapping table

Data manipulation

- Fetch from storage to memory

Main memory DRAM – dynamic random access memory - Need constant current to retain data - Content lost when power out - Uses non-volatile RAM that store basic information - bootstrap loader or BIOS	Memory and Storage Tools df: Storage usage stats mount: Mount existing file systems for access, or show currently mounted file systems iostat: Show storage I/O speeds free: Show memory usage hdparm/sdparm: Show I/O device stats and parameters
--	--

Hardware threads Each core may execute more than one thread - Hyper-threading - Shares certain physical resources - Presented to the OS as additional cores	Non-Uniform Memory Access (NUMA) - A processor + local memory - Accessing local memory is faster than remote memory - Speed is not uniform - Interconnect: communication medium that links Processors to processors, or memory
---	--

Data Organization

<https://www.cs.sfu.ca/~tzwang/teaching/cmpt454-fall26/slides/cmpt454-4-data-organization.pdf>

Data Organization

a collection of tables

- Page: basic unit of I/O - Typically not very large - 1 to 16KB
- Page size can be a tunable parameter
- May or may not be the same as physical device's page size
- Each page contains - A page header, A collection of slots, each slot stores a record

Record: collection of individual fields, for user-visible access

- Records can be variable or fixed length
- Each record identified by a unique Record ID

Fixed-Length Records: each field has fixed length

- Packed page - offset arithmetic to find a record
- Simple, but costly to move record around upon deletion
- Bit array, 1 bit to denote it contains a record, scan the bit array to locate record

Variable-Length Records: fields may have different lengths

- the number of fields is fixed, determined by schema
- Use an array of field offsets to locate fields

Page Format: Variable-Length Records

- Records could be of any size (though usually smaller than a page)
- Need to allocate just the right amount of space for a record
- Records may grow or shrink

Slotted page structure

- Each slot entry contains record offset
- Contain record length
- Flexible, but more complex
- Delete operation
 - Decrement the number of "valid" slots
 - Change the offset/size in slot entry to invalid
 - But cannot remove the slot entry - cuz RID will all change
- Insert operation
 - Try to reuse previously created slot
 - Need to scan existing slot entries

Use a Heap File

- fixed-size pages
- Pages are allocated and deallocated as the file grows and shrinks
- Pages are unordered in the heap file
- Hard to keep track of all pages and free space

Use a Linked List

- Header page and file name stored in well known locations

Use a Directory

- Identifies a page - includes page's free space or other info
- Has consist of directory pages (file it self or part of the main file)

Row-Oriented Storage

- Records stored 1 by 1 in page space
- Cheap to get all field
- Good for using UNION
- Many workloads only look at a few fields - calculate average for a column

Column Oriented Storage

- Getting multiple attributes is more complex, need additional I/O

Partition Attributes Across (PAX) - Hybrid NSM and DSM to get the best of both

- Each page partitioned into mini-pages

Data Access

<https://www.cs.sfu.ca/~tzwang/teaching/cmpt454-fall26/slides/cmpt454-5-data-access.pdf>

DB records must be brought to main memory for DBMS to access (read/write) them

Buffer Pool

- An array of fixed sized buffers (page frames)
 - Memory space to hold a page
 - Same size as a page
 - Pin count = number of requestors are using the page
 - Dirty bit - denote it is modified
- Page
 - 4 to 16KB

Handling a page request

1. Already in the buffer pool?
 - Yes – increment pin_count and return it, done — pinning a page
 - No – continue to the next step
2. Choose a page frame to accommodate the page
 - Vacant frame available
 - No - evict a page using page replacement policy
3. Load the page from storage to the chosen page frame
 - Increment pin_count by 1 and return
4. After use the page - unpin it by decrementing pin_count

Buffer Pool Structure

- Higher levels (e.g., transactions) use page IDs and record IDs to access data
- Need mapping between page ID and buffer pool page frame
- buffer pool is a global component - support concurrent access

Concurrent:

- concurrent hash table to support multiple threads
 - Fine-grained locks, Lock-free, A global mutex (simple but slow!)
- use multiple buffer pools
 - partition by table, page set, database. reduce contention on each hash table

Buffer Frame Replacement Policies

- Buffer pool usually $\lll$ data size
- Pages need to be evicted for new pages, impact on performance

Policies:

- Random
- FIFO
- Least recently used - LRU
 - Perfect tracking - Maintain a queue of pointers
 - Queue head == least recently used
 - pin_count becomes 0, enqueue at tail
- Clock replacement
 - approximate the Least Recently Used (LRU) algorithm
 - Each has a reference bit, initially = 0, when used, set to 1
 - Track history in Circular Organization (approximation)
- Most recently used MRU
 - Removes or evicts the page that was accessed most recently when the cache is full and a new page needs to be loaded
 - Repeated scan
- Sequential Flooding in LRU
 - repeated scans
 - Keep evicting the next needed page

DBMS Buffer Pool vs. OS Cache/Virtual Memory

- OS: files on disk, buffer cache in memory
- DBMS: database (files) on disk, buffer pool in memory
 - DBMS knows about the workload
- Can predict the access pattern for prefetching

Index

- a data structure technique used to speed up data retrieval operations by minimizing the number of disk accesses required to locate records
- Map search keys to values
- Search key can be values of a single or multiple fields
- Input = key, output = record or location of record
- Hash table, skip list or trees

Index operations

- Insert: establish key-value mapping
- Read/Get: retrieve value for a given key
- Update/Put: update the value of a given key
- Range Scan: return all values for a range of keys
 - Range can be [start key, end key] or [start key, number of records to scan]
 - Forward scan: scan from smaller to larger keys
 - Reverse scan: scan from larger to smaller keys
 - Typically not supported by hash tables

Index types

- Unique Index: One key maps to exactly one value, no duplicates allowed
- **Primary index**: index on a set of fields that include the primary key
 - Must be a unique index: no duplicates
 - No null values allowed
- Secondary index: index other fields

A primary index is built on a table's primary key and dictates the physical or primary ordering of data, whereas a secondary index is built on non-primary key columns to provide alternative lookup paths without changing the table's physical order.

Alternative for data entries in an index

1. A data entry k^* is an actual data record
 - Clustered,
 -
2. A data entry is a $\langle k, rid \rangle$ pair, where rid is the record id of a data record with search key value k
 - a clustered index only if the data records are sorted on the search key field. - Otherwise, the order of the data records is random
 -
3. A data entry is a $\langle k, rid\text{-list} \rangle$ pair, where $rid\text{-list}$ is a list of record ids of data records with search key value k .
 - a clustered index only if the data records are sorted on the search key field. - Otherwise, the order of the data records is random

Clustered vs. Unclustered Index

Clustered (grouped)

- Order in index == order in data file
- Better performance for scans
- Determines the physical order of data rows on the disk. Actual data rows are sorted and stored based on clustered index key values.
- A data file can be clustered on at most one search key, which means that we can have at most one clustered index on a data file.

Unclustered

- Order in index $\neq$ order in data file
- Slower - more random reads
- Nonclustered Index: Stored in a separate structure completely independent of the physical data rows. It does not rearrange the table's physical storage.

Suppose that Students records are sorted by age; an index on age that stores data entries in sorted order by age is a clustered index. If in addition we have an index on the gpa field, the latter must be an unclustered index.

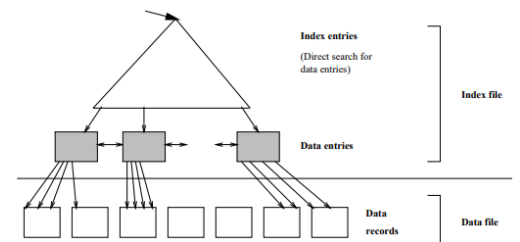
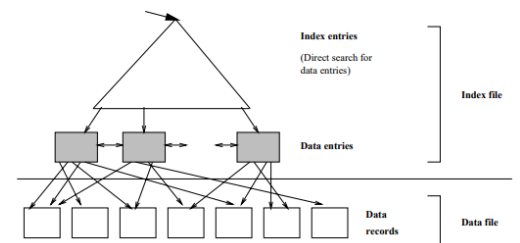


Figure 8.3 Clustered Tree Index Using Alternative (2)



Catalogue - Information about the DBMS, each table, and index

- Implemented as relations, data dictionary
- System wide information - page size, buffer pool size
- Table specific information - table name, ID, constraints, available indexes
- Index specific information - name structure, index key
- Statistics - number of tuples for each table

Tree Structures

<https://www.cs.sfu.ca/~twang/teaching/cmpt454-fall26/slides/cmpt454-6-trees.pdf>

Primary or secondary indexes

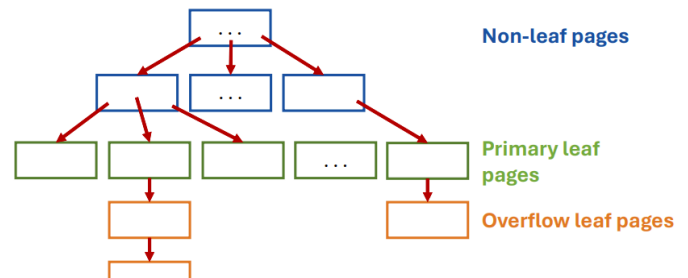
- Use trees or hash tables

Range Search

- High cost of binary search (random)

Index Sequential Access Method (ISAM)

- Index file to actual records
- ISAM tree
 - Allocate leaf pages sequentially, sorted by key
 - Allocate index (non-leaf) pages
 - Allocate space for overflow leaf pages



To search - start from root, and key compare

To insert - find leaf node

Delete - find and delete leaf node - deallocate node if empty

- Lock non-leaf pages - not dynamically addressing

B+ tree or B-Tree

- Dynamic structure
- Each node occupies a storage page
- Balanced: all paths from root to the lowest level have the same length

Insert/delete complexity $\log_F N$, F - number of child pointers in internal node, N = # leaf nodes

- Minimum 50% occupancy
- Data entries are only stored in leaf nodes

Root node - Initially a leaf node, become internal node as tree grows

Internal node - store pointer (addresses) and separator keys

- compare target key and separator key during traversal

Leaf node - store M keys and M payloads

- Payloads can be data or address, variable length, can chained together

Internal vs. leaf nodes

- Internal nodes usually hold more entries than leaf nodes do
- Internal nodes change more slowly than leaf nodes

Node Format

- Idea 1: Rearrange key upon insert, not efficient to search
- Idea 2: only rearrange fixed size slots, binary search

B-Tree Insert - Find the leaf, put data

- If enough space - done
- If not - split leaf to new node, redistribute entries, insert the middle key to parent.

B-Tree Delete - Find the leaf

- Done if L is at least half full
- Else: redistribute or merge (in practice: not done right away)
- Borrow from siblings. (merge: combine with sibling to get a single node)

A stack can be used during traversal to avoid re-traversal

Inheritance used for implementing node types in memory

Latches in Database Systems

- For lock table in lock manager
- For buffer page frames
- Embedded page frame header, in-memory B-tree node header

2 modes - reader - shared, writer - exclusive ==== S/X = shared / exclusive

- Exclusive - 1 thread only

Protecting B-Tree

- Cannot be modified simultaneously by another thread while a thread is using
- Latch for each node
- Protect pointers
- Must not allow: thread T2 to deallocate node N2 when T1 follows right pointer at root node to access N2
 - Lock coupling

Lock coupling

- Take S/X latches for individual node before access
- Unlatch parent node when child node is safely latched
 - Will not split or merge for updates
 - Safe when: Not full (for inserts), not less than half full (for deletes)

Search for a single key (point query):

- Start by Shared-latching root node
- Acquire Shared-latch on child node
- Release Shared-latch on parent node

Goal of latch coupling is to release parent latches as early as possible

- Search - release parent right away, most at most 2 latches
- insert/delete - x-latch the child: if child is safe, release all ancestors; if not, keep all

Insert:

- X-latch on affected nodes
- Splits/merge may propagate upward to the root

Alternative 1: For insert/delete/update, X-latch all nodes during root-to-leaf traversal

- Unlatch parent node if child node will not split after insertion
- Low concurrency - serial on the root level

Alternative 2: For insert/delete/update, S-latch all nodes as usual, upgrade to X-latch as needed

- Complex code
- Unlatch parent node if child node will not split after insertion
- May deadlock when upgrade to X-latch

Other Alternative: split early, update mode, optimistic locks, lock-free and B-link trees

B-Tree Bulk Loading - create a new index

Alternative 1: create an empty tree and do inserts to it

- Easy, 1 record at a time, slow(high overhead)

Alternative 2: build the tree bottom-up

- Sort the data
- Create an empty root, points its left most pointer to the first leaf node
- Add 1 entry in the root node for each node - start from first leaf node - Fill in all the leaf node
- Split the root node if needed (build up)

- keep adding keys to the right-most internal node just above the leaf level, split and grow tree as needed
- Fast on tree traversal

B-Tree Key Compression

Fit more keys into each node

- Desired: high fanout (child pointers per node to lower level)
- Fewer nodes (lower height) → reduced I/O

Prefix key compression: Extract the common prefix and store only the unique suffix for each key

- Sorted keys in nodes tend to have the same prefix

Suffix truncation: Store only the prefix of each key that is “enough” to route traffic in internal nodes

- Inner node keys are only separators

B-tree variants

- Optimization for cache access, memory, storage
- lock-free/SIMD instructions
- In-memory adaptations

Skip Lists

<https://www.cs.sfu.ca/~tzwang/teaching/cmpt454-fall26/slides/cmpt454-7-skip-lists.pdf>

Sorted Linked list

Skip List: Linked List with Shortcuts

- Multiple levels of sorted linked lists
- Tower: Nodes in higher levels allow skipping key ranges
- Lowest level, includes all keys in sorted order
- $O(\log N)$ levels in total

Basic idea on search (key):

- If current key == key: done
- If not at lowest level
 - If key < next key: go down, repeat
 - If key >= next key: go right, repeat
- If already at the lowest level, do linear search

Insert and Delete

- Might rearrange the entire list
- Expensive to ensure each level has exactly $\frac{1}{2}$ elements in the lower level
- Randomized data structure: the same sequence of inserts and deletes may result in two different skip lists

Insert to the lowest level: same as inserting into a linked list

- Then roll dice to decide whether to insert into higher levels
- Need to remember the path leading to the lowest level (visited nodes) (add another higher level)

Range Scans

- Forward scan - easy with singly linked skip list
- Reverse scan - doubly linked skip list

Remembering Traversal Path

- Maintain a stack/list/array when traversing the skip list
 - Useful for reverse scan, insert and delete operations
- Reverse scan - find start key and move backwards

Reverse Scan with a Stack

- If go over, can easily go back

Building up Towers with a Stack

- Repeat the pop-build process until all levels are built
- If the height of the skip list grows, then link with head and tail nodes.

Concurrency

Lock-based

- Lock adjacent nodes during insert (similar to latch coupling)
- Lock whole levels during traversal (Project Milestone 2)

Lock-free: use atomic instructions (compare-and-swap, fetch-and-add, atomic-swap, etc.)

- Can be easy or difficult depending on skip list design
- Typically gets higher performance than lock-based designs

Doubly-linked - lock free is hard

Singly linked - lock free is easy with compare and swap

Compare_exchange (current_value, expected, desired)

- Current_value = pointer
- Desired = new value to be written
 - If current_value == expected -----> current_value = desired; return true;
 - Else -----> expected = current_value; return false;
- Check return value and handle either case
- Repeat until CAS succeeds: do { } while() loop

Lock-Free Skip List

- insert to the bottom level first, then build up the tower gradually from bottom up

Insert:

1. Traverse to the insert position, at the lowest level (no latch)
2. Prepare a new node
3. Insert in the lowest level with CAS,
 - If failed, move backwards and find the correct position
 - New value is visible, start to build the tower with same CAS approach

Delete: Concurrency issue

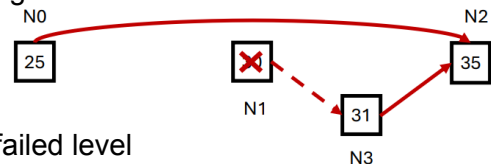
- If insert and delete at the same time with 2 threads, insert thread don't know that the node is deleted, insertion will lost because there is no node link to the new inserted node

Solution

- Use the first bit of the node as "delete bit"
- Delete: toggle deleted bit (change to 1) - logically deleted, can still be traversed, but will skipped
- Insert: expect predecessor's deleted bit is 0 - will stop inserting when it is 1

CAS(&N1.next, 0 | N2, N3)

- N1 = node to delete, N3 node to insert
- CAS succeeded → proceed to the next higher level
- CAS failed → neighbours changed, re-traverse and retry the failed level



Problem:

- Too much pointer changing, cache misses
- Many IO, one node per key
- Waste space

Optimization1: Co-locate all nodes in the same tower

- One big node per tower, multiple pointers to the next nodes on the right
- Reduced cache misses

Optimization 2: Unrolled list

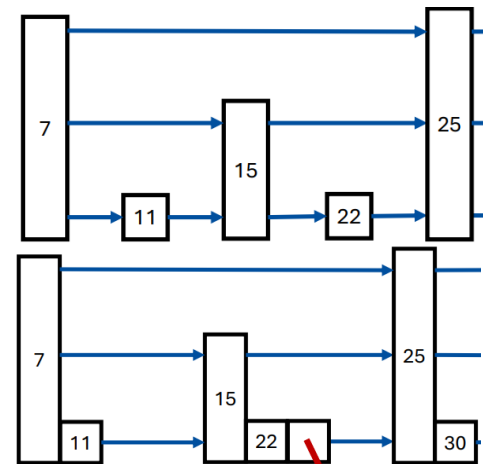
- Link big (fixed-size) chunks of memory, instead of individual nodes
- Each chunk contains multiple nodes
- Free space available for inserts (need space management)
- Further reduce cache misses

Optimization 3: Determining tower height

- Each chunk is the same size, the right most 2 chunks should have extra free space next or below (can be used for more storage)

while (uniform_random(0, 1) == 0): // keep building up the tower

- Many calls into the random number generator (RNG)
 - Potentially many memory writes
 - Synchronization overhead



Skip Lists vs. B+-Trees

- Both order preserving
- Skip lists adopted by some DBMSs and popular key-value stores (KVS) as well
- B+-trees - More optimized for storage, more cache efficient than skip lists

- Hard to be latch-free
- Easy to support both forward and reverse range scans
- Skip lists
 - Not friendly to disk/storage – not optimized for locality
 - Easy to be latch-free
 - Supporting reverse range scan is non-trivial

Hash Tables

<https://www.cs.sfu.ca/~tzwang/teaching/cmpt454-fall26/slides/cmpt454-8-hash-tables.pdf>

A collection of buckets, each bucket holds keys (and associated payload) that are hashed into the same bucket using a hash function H

- Hash function maps key to an index into bucket number
- Each buckets contain multiple data - data collision
- Good for point queries, Bad - can't do range scans

Static Hashing

- Fixed number of buckets
- Each bucket consists of one or more pages
- Each bucket page has fixed capacity
 - No space in single page: create overflow pages chained together per bucket

Hash function must evenly distribute keys into different buckets

- Chains of overflow pages can become long - performance issues
- Waste space when data size reduced significantly - empty buckets
- High cost to re-organize/expand by increasing the number of buckets

Dynamic Hashing - NOT on midterm